

KNOWLEDGE GRAPHS

GRAPHRAG VS VECTOR RAG

Why Vector RAG Fails (And GraphRAG Fixes It)

Ask Vector RAG "Find invoice #402" → Perfect. Ask "What are the main themes across 1,000 PDFs?" → Complete Failure. Here is why.

Text Corpus

Entity
ExtractionKnowledge
GraphCommunity
Summaries

GraphRAG

**Search****Paradigm Shift:** Vector search finds local text chunks. GraphRAG connects dataset-wide global relationships.

“ **THE INDEXING ANALOGY:** Vector Search is like a **Magnifying Glass** (finds local quotes). GraphRAG is like a **Table of Contents** (understands full structure).

Swipe → Step-by-step RAG breakdown

STEP 01

LOCAL SIMILARITY

Step 1: How Vector RAG Works

Vector RAG slices text into fixed chunks, computes dense vector embeddings ($e_i \in \mathbb{R}^d$), and retrieves top-K nearest neighbors via Cosine Similarity.

STANDARD VECTOR RAG RETRIEVAL FLOW

VECTOR_RAG.PY

```
chunks = chunk_document(text, size=512, overlap=50)
query_vec = embedding_model.encode("Who is the CEO of Acme?")
results = vector_db.search(query_vec, top_k=5)
```

$$\text{Cosine Similarity}(q, d) = (q \cdot d) / (||q|| ||d||)$$

Where Vector RAG Excels

- ✓ **LOCAL FACT RETRIEVAL:** "What is the warranty policy on Product X?" → Fetches the exact 500-token chunk instantly from vector DB.

“ **CRUCIAL:** Vector RAG excels at explicit, localized fact-finding queries where answers exist inside a single text chunk.



✖ STEP 02

FAILED QUERY SCENARIOS

Step 2: Why Vector RAG Fails

What happens when you ask complex, global queries that span hundreds of documents?
Vector search breaks down completely.

✂ 2 Failed Query Scenarios

- ✖ 1. GLOBAL SUMMARIZATION: "What are the top 5 recurring supply chain bottlenecks across all 500 vendor audit reports?" (No single chunk contains this!).
- ✖ 2. MULTI-HOP REASONING: "How does Person A's company connect to Person B's offshore account?" (Requires multi-edge relationship traversal).

🔻 Top-K Blindspot

Retrieving Top-5 chunks misses big picture context spread across 1,000 files.

💡 Core Limitation

Vector embeddings capture semantic similarity, NOT structural relationships.

💡 **CRUCIAL:** Vector search cannot summarize a dataset because embeddings don't build dataset-wide macro trends.

STEP 03

RRF & RERANKING

Step 3: Hybrid Search (BM25 + Vector)

Before GraphRAG, production RAG systems combine Sparse Keyword Search (BM25) with Dense Vector Search using Reciprocal Rank Fusion (RRF).

HYBRID SEARCH WITH RRF & CROSS-ENCODER

HYBRID.PY

```
bm25_hits = bm25_index.search(query, top_k=20)
vector_hits = vector_db.search(query_vec, top_k=20)
fused_hits = rrf_combine(bm25_hits, vector_hits)
final_hits = cross_encoder.rerank(query, fused_hits, top_k=5)
```

$$\text{RRF Score}(d) = \sum \frac{1}{k + r_m(d)}$$

Dual Search Strengths

A BM25 Keyword: Catches part numbers, code symbols & IDs (

ERR-9021).

Vector Similarity: Catches synonyms, intent & fuzzy conceptual meaning.

66 CRUCIAL: Hybrid Search fixes missing exact keywords, but STILL struggles with global synthesis across 1,000s of files.



STEP 04

GRAPH EXTRACTION

Step 4: Enter GraphRAG

GraphRAG uses an LLM to transform unstructured text documents into a structured Knowledge Graph of Entities & Relationships.

3-Step Graph Generation Pipeline

-  **1. ENTITY EXTRACTION:** LLM identifies Subject, Object & Types ("Samara" → Person , "Resnur AI" → Org).
-  **2. RELATIONSHIP EXTRACTION:** LLM extracts explicit relationship claims ("Samara" → founded → "Resnur AI").
-  **3. GRAPH SYNTHESIS:** Constructs a multi-node, multi-edge directed Knowledge Graph.

 **WEB OF KNOWLEDGE ANALOGY:** Turning thousands of isolated pages into an interconnected web of nodes (people, places, concepts) and explicit directed edges.

❗ CRUCIAL: GraphRAG replaces isolated text chunks with an interconnected graph of real-world entities and relationship claims.



STEP 05

LEIDEN CLUSTERING

Step 5: Community Summaries

How does GraphRAG answer dataset-wide global questions? It runs graph clustering algorithms (Leiden / Louvain) to create hierarchical summaries.

Hierarchical Clustering Execution

-  **1. COMMUNITY DETECTION:** Leiden algorithm clusters closely related entity nodes into hierarchical graph communities.
-  **2. COMMUNITY SUMMARIZATION:** LLM generates pre-computed summaries for every cluster at multiple granularities (Level 0, 1, 2).
-  **3. GLOBAL QUERY SEARCH:** To answer "*What are top themes?*", GraphRAG queries pre-computed community summaries in parallel!

“ CRUCIAL: Pre-computing community summaries turns expensive full-dataset scans into fast, parallelized map-reduce summaries.

STEP 06

QUERY ENGINES

Step 6: GraphRAG Query Modes

GraphRAG supports dual query engines tailored to the question's granularity:

📍 Local Search (Entity)

Query: "What projects did Samara build at Resnur AI?"

Finds node "Samara" → traverses 1-hop & 2-hop edges → compiles text chunks.

🌐 Global Search (Dataset)

Query: "What are key technical challenges in modern AI deployment?"

Scans Level-1 Community Summaries → runs parallel LLM map-reduce.



DUAL ADVANTAGE: Local GraphRAG answers multi-hop entity questions; Global GraphRAG answers high-level thematic questions across 1,000s of files.

“ CRUCIAL: Local Search traverses explicit graph edges; Global Search aggregates community summaries in parallel.



STEP 07

ARCHITECTURE DECISION

Step 7: The RAG Decision Matrix

Comparing Vector RAG, Hybrid Search, and GraphRAG across cost, latency, and query capabilities:

FEATURE / CAPABILITY	VECTOR RAG	HYBRID (BM25+VEC)	GRAPHRAG
Exact Code / ID Match	✗ Poor	✓ Excellent	● Moderate
Local Fact Retrieval	✓ Excellent	✓ Excellent	✓ Excellent
Multi-Hop Relationships	✗ Fails	✗ Fails	✓ Excellent
Global Summaries / Themes	✗ Fails	✗ Fails	✓ Excellent
Indexing Cost & Time	● Low (\$)	● Low (\$)	● High (\$\$\$)
Query Latency	● ~200ms	● ~350ms	● ~1-3s

“ **CRUCIAL:** Vector RAG for simple Q&A, Hybrid for Enterprise Search, and GraphRAG for complex analytics & discovery.

 STEP 08

ENTERPRISE PIPELINE

Step 8: Production Hybrid GraphRAG

In production enterprise AI systems, top teams don't choose just one—they layer all 3 paradigms together with an LLM Router!

ENTERPRISE HYBRID ROUTED RAG PIPELINE

ENTERPRISE_RAG.PY

```
if query_type == "exact_keyword":
    results = bm25_search(query)
elif query_type == "local_fact":
    results = hybrid_vector_rerank(query)
elif query_type == "global_synthesis":
    results = graph_rag_community_search(query)
return llm_generate_answer(query, results)
```

Router-Driven RAG Execution

-  **DYNAMIC ROUTING:** A lightweight Router LLM classifies incoming queries and dispatches them to the optimal search engine.

“ **CRUCIAL:** The ultimate enterprise architecture uses a Router LLM to dispatch queries to the optimal RAG engine.

ONE-LINE MENTAL MODEL

CHEAT SHEET

The Complete RAG Landscape

The ultimate RAG cheat sheet for AI System Design & Architecture:

THE 3-TIER RAG CHEAT SHEET

-  **Vector RAG:** Dense Similarity ($e_i \in \mathbb{R}^d$) · Fast · Best for Local Fact Q&A
-  **Hybrid Search:** BM25 + Vector + Cross-Encoder · Best for Enterprise Code/IDs
-  **GraphRAG:** Knowledge Graph + Community Summaries · Best for Global Themes

⚡ THE ONE-LINE MENTAL MODEL:

Vector = Magnifying Glass | Hybrid = Index | GraphRAG = Table of Contents

1. Save This Post

Keep this reference guide handy for RAG system design & AI interviews.

2. Reshare

Help your fellow engineers understand modern RAG architectures.

 **Level Up Your AI Knowledge:** Save this post, Reshare & follow resnur.ai for daily AI architecture carousels!